SQL-Based Vulnerability Assessment of Organizational Data

Project Description

In this project, I acted as a security professional investigating potential vulnerabilities within a large organization. I utilized SQL filters—specifically `AND`, `OR`, and `NOT`—to query the `log_in_attempts` and `employees` tables. My goal is to isolate suspicious login patterns, such as after-hours failures and international attempts, and to audit employee data across various departments to ensure system security and compliance.

Retrieve after hours failed login attempts

To identify potential security risks occurring outside of standard business hours, I queried the `log_in_attempts` table to isolate failed login attempts that took place after 18:00.

SQL

```
SELECT *
FROM log_in_attempts
WHERE login_time > '18:00' AND success = 0;
```

	event_id	username	login_date	login_time	country	ip_address	success
0	1	jdoe	2022-05-08	18:30	MEX	192.168.1.5	0
1	3	jdoe	2022-05-09	20:45	MEX	192.168.1.5	0
2	5	vlee	2022-05-10	23:15	UK	192.168.1.45	0

How the query works

This query uses the AND operator to ensure that the results strictly meet two specific security criteria:
- **WHERE login_time > '18:00'**: This filter targets the login_time column to retrieve only those records where the login attempt occurred after 18:00 (6:00 PM).
- **AND**: This logical operator requires both conditions to be true for a record to be included in the results.
- **success = 0**: This part of the filter focuses on the success column, where a value of 0 (or FALSE) represents a failed login attempt.

By filtering for both an off-hours timestamp and a failed status, this query highlights suspicious activity that may indicate unauthorized access attempts or brute-force attacks when the office is typically closed.

Retrieve login attempts on specific dates

To investigate a suspicious event that occurred on 2022-05-09 , I queried the `log_in_attempts` table to review all activity from that day and the preceding day, 2022-05-08.

SQL

```
SELECT *
FROM log_in_attempts
WHERE login_date = '2022-05-09' OR login_date = '2022-05-08';
```

	event_id	username	login_date	login_time	country	ip_address	success
0	1	jdoe	2022-05-08	18:30	MEX	192.168.1.5	0
1	2	asmith	2022-05-09	09:15	CAN	192.168.1.12	1
2	3	jdoe	2022-05-09	20:45	MEX	192.168.1.5	0

How the query works

This query utilizes the `OR` operator to broaden the search criteria across two specific dates:
- **`WHERE login_date = '2022-05-09'`**: This condition filters the `login_date` column for any records matching the exact date of the reported suspicious event.
- **`OR`**: This logical operator allows the query to return a record if **either** of the conditions is met. Unlike `AND`, it does not require both dates to be true for a single row.
- **`login_date = '2022-05-08'`**: This second condition includes the day before the event, providing additional context for the investigation.

By using this filter, the resulting dataset captures all login activity—both successful and failed—within that 48-hour window, which is essential for identifying patterns or unauthorized access leading up to the security incident.

Retrieve login attempts outside of Mexico

To investigate potential unauthorized access from international locations, I queried the `log_in_attempts` table to identify activity originating from countries other than Mexico.

SQL

```
SELECT *
FROM log_in_attempts
WHERE NOT country LIKE 'MEX%';
```

	event_id	username	login_date	login_time	country	ip_address	success
0	2	asmith	2022-05-09	09:15	CAN	192.168.1.12	1
1	4	rbrown	2022-05-10	11:00	USA	192.168.1.20	1
2	5	vlee	2022-05-10	23:15	UK	192.168.1.45	0

How the query works

This query utilizes the NOT operator to exclude a specific geographic region from the results:
- **WHERE NOT**: This operator reverses the logic of the condition that follows it, ensuring that only records which do **not** match the criteria are returned.
- **country LIKE 'MEX%'**: This part of the filter targets the country column. The LIKE operator combined with the % wildcard allows the query to capture any variation of "Mexico" (such as "MEX" or "Mexico").

By filtering out all login attempts from Mexico, this query allows the security team to focus strictly on international traffic, which is a key step in identifying suspicious remote access patterns.

Retrieve employees in Marketing

To investigate potential security risks within specific departments, I queried the `employees` table to identify all staff members assigned to the Marketing team.

SQL

```
SELECT *
FROM employees
WHERE department = 'Marketing';
```

	employee_id	device_id	username	department	office
0	101	dev_901	jdoe	Marketing	North-101
1	105	dev_905	mgarcia	Marketing	North-105

How the query works

This query uses a basic filter to isolate a specific segment of the workforce:
- `SELECT *`: This retrieves all columns from the `employees` table—including `employee_id`, `device_id`, `username`, `department`, and `office`—providing a complete profile for each individual.
- `WHERE department = 'Marketing'`: This condition filters the `department` column to return only the records where the value matches 'Marketing' exactly.

By focusing on this department, the security team can review the access levels and machine permissions specifically for Marketing employees to ensure they align with organizational security policies.

Retrieve employees in Finance or Sales

To investigate potential security risks across multiple departments, I queried the `employees` table to identify all staff members in either Finance or Sales.

SQL

```
SELECT *
FROM employees
WHERE department = 'Finance' OR department = 'Sales';
```

	employee_id	device_id	username	department	office
0	102	dev_902	asmith	Finance	South-202
1	103	dev_903	rbrown	Sales	East-303

How the query works

This query uses the `OR` operator to capture records from two different departments simultaneously:
- **`WHERE department = 'Finance'`**: This condition filters for any employee belonging to the Finance team.
- **`OR`**: This operator allows the query to return a record if it matches **either** the

Finance condition **or** the Sales condition.
- `department = 'Sales'`: This condition expands the search to include the Sales team.

By using this filter, the security team can efficiently audit access permissions for these two high-traffic departments in a single view.

Retrieve all employees not in IT

To investigate potential security risks across multiple departments, I queried the `employees` table to identify all employees who are not part of the IT department. This helps in assessing the access levels and potential vulnerabilities of users who do not have administrative or technical roles.

SQL

```
SELECT *
FROM employees
WHERE NOT department = 'IT';
```

	employee_id	device_id	username	department	office
0	101	dev_901	jdoe	Marketing	North-101
1	102	dev_902	asmith	Finance	South-202
2	103	dev_903	rbrown	Sales	East-303
3	105	dev_905	mgarcia	Marketing	North-105

How the query works

This query uses the `NOT` operator to filter the results based on exclusion:
- `SELECT *`: This retrieves all columns from the `employees` table, including `employee_id`, `device_id`, `username`, and `office`.
- `FROM employees`: This specifies the table containing the personnel data.
- `WHERE NOT department = 'IT'`: This condition looks at the `department` column and returns every row where the value is **not** 'IT'.

By applying this filter, the security team can focus their investigation on staff members in departments like Marketing, Finance, and Sales, ensuring that their machine permissions and login activities are appropriate for their non-technical roles.

Summary

Through the application of SQL filters, I successfully identified several key areas for further security review. I isolated failed login attempts occurring after 18:00 and on specific dates associated with suspicious events. Additionally, I filtered for international login attempts and categorized employees by department to better understand the organization's access landscape. These queries provide a foundational dataset for identifying unauthorized access and refining the organization’s security protocols.